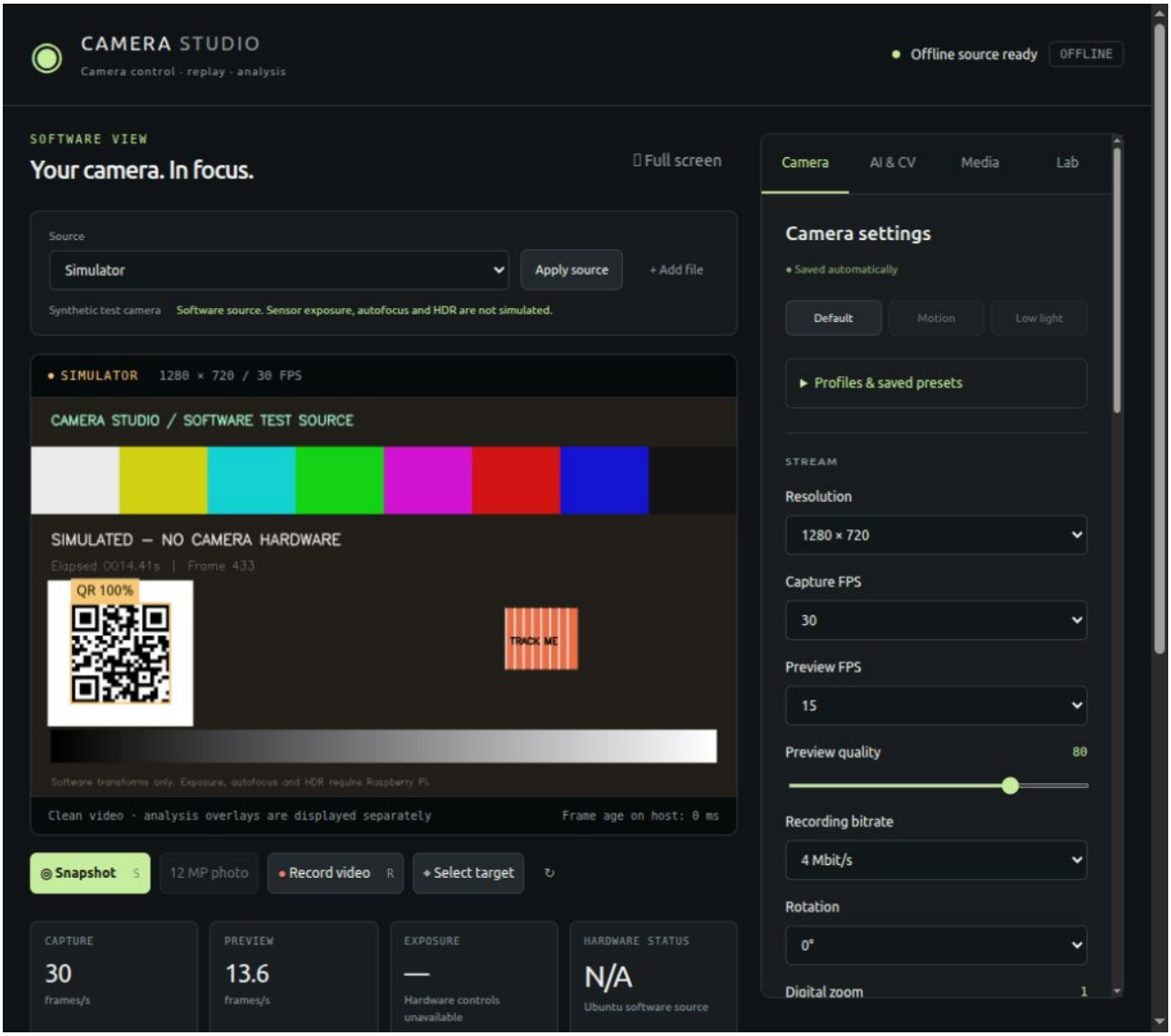
CAMERADRONE

Camera Studio engineering guide

Ubuntu host application - Raspberry Pi camera - offline development - airborne video preparation

Release 0.3 / 3 November 2026

Real Pi 5 capture, AI, recording and single-encoder H.264/RTP fanout are documented with measured Ethernet decoder results. Radio hardware, optical latency and Pi 3 A+ performance remain unverified.



Synthetic preview only. No real camera imagery or credentials are included.

Guide contents

01 Quick start

02 Setup and deployment

03 Application workflow

04 Architecture and API

05 Airborne transport

06 Validation and limits

07 Current Pi 5 regression

08 Real camera RTP validation

09 Installation and boot strategy

10 Hardware roadmap

11 Third-party components

12 Public release workflow

Source of truth

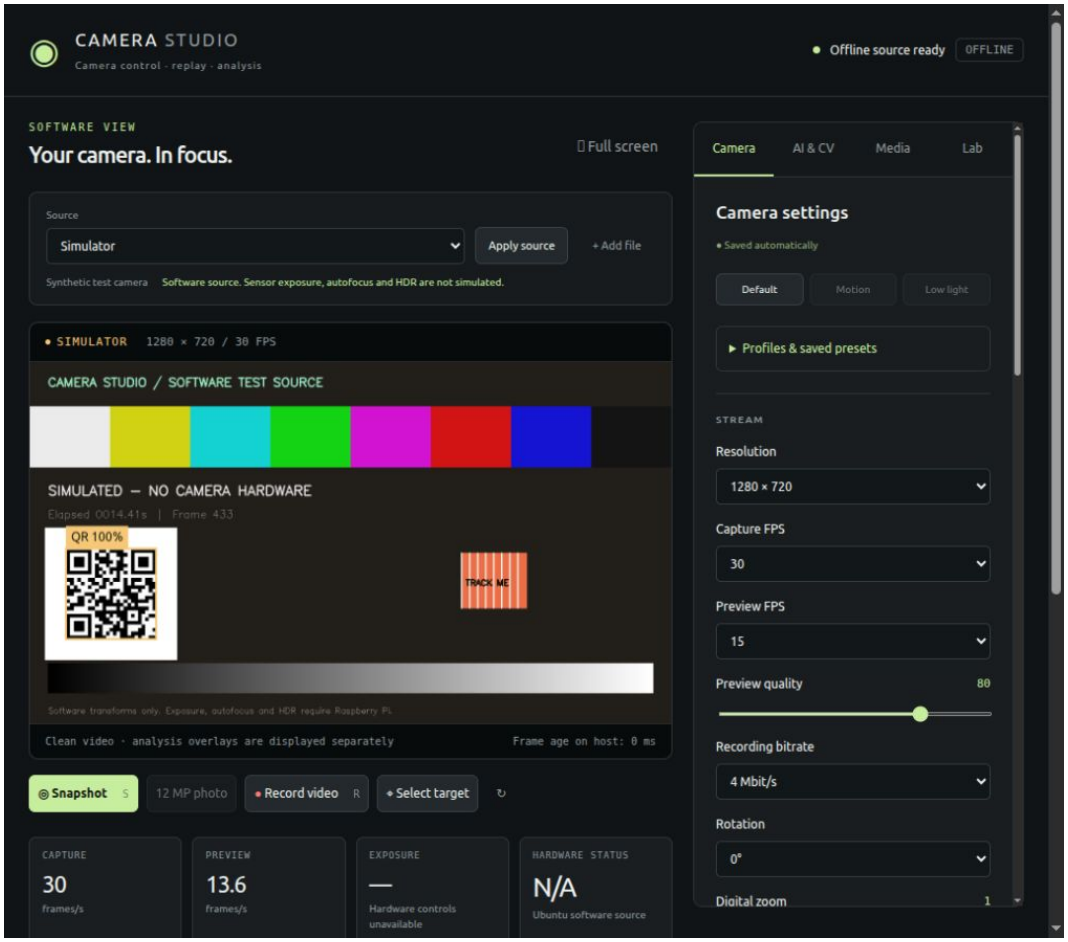
This guide is generated from the Markdown documents in the repository. Shell examples use the repository working directory. Use your virtualenv Python where appropriate. Local addresses shown in examples are placeholders; private runtime files and test recordings are intentionally absent.

Rebuild

Install ReportLab and system DejaVu fonts, then run python3 tools/build_pdf.py. The Markdown documents retain clickable relative links and Mermaid diagrams; the PDF includes simplified text schematics.

CameraDrone / Camera Studio

An English-language camera control application for an Ubuntu host and a Raspberry Pi CSI camera. Develop and test the application **without a Raspberry Pi** using a synthetic camera or recorded inputs. Capture, ISP and inference run on the Pi when a real camera is selected; offline processing runs on Ubuntu.



Camera Studio using a generated test source

Start without hardware

On Ubuntu 22.04 with Python 3.10 or newer:

```
sudo apt-get update
sudo apt-get install -y python3-venv gstreamer1.0-tools gstreamer1.0-plugins-base
gstreamer1.0-plugins-good gstreamer1.0-plugins-bad gstreamer1.0-plugins-ugly
gstreamer1.0-libav
python3 -m venv .venv
.venv/bin/pip install -r requirements-host.txt
.venv/bin/python camera_app/fetch_models.py
.venv/bin/python camera_app/host.py
```

Open **http://127.0.0.1:8765/**. A fresh installation starts in **Simulator** mode. No microSD, SSH connection, camera driver or Pi is needed. To install a user service, run `.venv/bin/python tools/setup_host.py --install-service`. This generates paths for the current checkout and Python interpreter.

What works

Feature	Raspberry Pi	Simulator / Replay
Preview and browser overlays	CSI video via host proxy	Generated frames or local file
Resolution, FPS, preview quality	Camera pipeline	Software output transform
Exposure, gain, AWB, focus, denoise, processing HDR	Camera/ISP controls, subject to capabilities	Disabled; explicitly not simulated
Brightness, contrast, saturation, sharpness, zoom, rotation	ISP/camera controls	Approximate software transforms
Snapshots and video	Pi storage, H.264 MP4 + metadata	Ubuntu storage, JPEG / H.264 MP4 + metadata
Full-sensor photo	IMX708: 4608 x 2592; pauses preview	Unavailable
Objects / faces / QR / motion / ROI tracking	Pi CPU	Ubuntu CPU
Presets, profile selection and exports	Controlled from Ubuntu	Same host application
File import, playback pause/seek, analysis reports	Host lab remains available	Supported locally
Camera H.264/RTP	Pi 5 Ethernet implemented	One encoder, recording fanout, native host decode
Airborne radio video	RF hardware pending	WFB-ng integration contract and diagnostic tools

The tested sensor is **Sony IMX708**, identified by the camera stack; the board label supplied was Arducam UC-b47 rev B. Do not infer an AI accelerator from that board label. The application uses CPU vision. The separate GigE Vision camera is neither discovered nor configured.

Verification status

The current version passed 18 functional Pi camera checks, six RTP lifecycle/guard checks and 19 offline tests. A corrected 30-minute real-camera Ethernet run decoded 54071 frames at 30.014 FPS, with no lost/late RTP packets and a 78.651 ms maximum decoded-frame gap. The initial failed receiver run and its fix remain documented in the transport report; earlier capture/recording regressions remain in the hardware report. Pi 3 A+, RF performance and optical end-to-end latency remain unverified. See validation evidence and limits.

Documentation

- Setup, deployment and recovery
- Application usage and offline workflow
- Architecture, API and data
- Airborne video and driver plan

- Pi 3 A+ migration and next hardware checks
- Public release workflow
- Validation, third-party notices, PDF guide

Development checks

```
python3 -m unittest discover -s tests -p 'test_*.py' -v
python3 airlink/diagnose.py
```

Use the configured virtualenv interpreter instead of `python3` if needed. Tests use synthetic inputs, temporary directories and localhost sockets. They never connect to the Pi. GitHub Actions runs this suite on Ubuntu 22.04. Historical hardware checks are separate scripts and require an explicitly selected real camera.

Project layout

```
camera_app/       Host UI, source manager, camera API, Pi and offline engines, vision
camera_app/models/ Pinned CPU model and upstream license
airlink/          H.264/RTP sender, ground receiver, profiles, read-only inventory
tests/            Offline integration tests and separate historical hardware checks
tools/            Portable host setup, optional test fixtures, PDF/publication tooling
docs/             English Markdown documentation and synthetic UI screenshot
output/pdf/       Rendered technical guide
```

Runtime secrets, recordings, local inputs, known host keys, logs, caches and intermediate build files are excluded. The Pi API uses a private shared token over a trusted LAN; the host binds to localhost. This is a local development application, not an Internet-facing multiuser camera service.

See real camera RTP validation, air-link setup and installation bundle / boot strategy. The software archive is reproducible and sanitized; it is not a bootable OS image.

Setup, deployment and recovery

Ubuntu host

Use the installation commands in the root README. The host needs Flask, Requests, NumPy, OpenCV and GStreamer. Picamera2 is **not** imported by the host. `opencv-python-headless` supplies its own face cascade; Debian `opencv-data` is also supported. GStreamer `x264enc`, `mp4mux`, `h264parse`, RTP plugins and `avdec_h264` are required for recording and the transport bench.

Start in foreground with `.venv/bin/python camera_app/host.py`. It binds to 127.0.0.1:8765. `CAMERA_HOST_PORT` can override the port, and `CAMERA_HOST_DATA` can select another private data directory. Use one host instance per data directory.

Optional user-session startup:

```
.venv/bin/python tools/setup_host.py --install-service
systemctl --user status camera-studio.service
journalctl --user -u camera-studio.service -n 80
bash start-studio.sh
bash stop-studio.sh
```

The setup script generates the user service and application-menu shortcut using the current paths. It does not contact the Pi. The service is tied to the host user's session by default. `start-studio.sh` starts an installed service or runs Python in the foreground; for a virtualenv foreground launch, use its Python explicitly.

A first run generates `camera_app/runtime.json`, mode 0600, with a random API token and default `http://camera.local:8081`. The UI lets you enter a private Pi address. Simulator and Replay never silently fall back to a Pi. The selected source is remembered, so an installation last used in Pi mode can show unavailable until Simulator is selected.

Raspberry Pi OS and camera

The reference system used Raspberry Pi OS with Picamera2 and an IMX708 camera on Pi 5. Ubuntu 22.04 is the **host** OS, not the recommended replacement for the Pi camera stack. Install Raspberry Pi OS to microSD using Raspberry Pi Imager, configure your own username, SSH and network, then boot the Pi. Keep storage connected for the entire operating session.

On the Pi, install camera-stack dependencies from the OS repositories:

```
sudo apt-get update
sudo apt-get install -y python3-picamera2 python3-flask python3-requests python3-opencv
opencv-data ffmpeg
rpicam-hello --list-cameras
```

Confirm the model and camera index before deployment. Older OS releases may expose `libcamera-*` names instead of `rpicam-*`; use a supported Raspberry Pi OS camera stack. Only one process can own the CSI camera. Stop an old `rpicam-vid`, OpenHD camera service or another capture application before starting this backend.

On Ubuntu, establish and verify the Pi SSH host key by a normal SSH login. Use your own username/private address; do not disable host-key checks. The helper uses normal SSH known_hosts, or `pi_known_hosts` in this checkout when that private file exists.

```
export PI_TARGET=youruser@192.168.1.20
bash deploy-studio.sh
```

The script verifies model checksums, transfers source/models/private shared token, generates a Pi user service, and starts it. It preserves `camera_app/data/` and never copies host input videos. Dependencies must already be installed. Review local source before redeployment and repeat the hardware regression after camera/backend changes. The current Pi 5 regression is documented separately.

On the Pi, enable boot startup without an SSH login:

```
sudo loginctl enable-linger "$USER"
systemctl --user status camera-backend.service
journalctl --user -u camera-backend.service -n 80
```

On Ubuntu, choose **Raspberry Pi**, enter `http://192.168.1.20:8081`, then **Apply source**. The token is shared automatically by the deployment script. If you regenerate the host token, redeploy the matching runtime file to the Pi. Avoid publishing that file.

Storage and boot

This release uses a normal bootable microSD on the Pi. Network boot and a USB boot image are future packaging work, not prerequisites for the host simulator. A bare board with only Ethernet and camera cannot serve video unless its EEPROM boot configuration and a working network-boot server provide the OS. The passive `diagnose-netboot.sh` helper is historical troubleshooting, not a complete network-boot installer.

Do not remove storage while the Pi is running. Use a clean shutdown before physically switching it off. Neither a host service nor this repository magically supplies a Pi OS image.

Recovery

• **Source unavailable:** select Simulator to continue locally; for Pi mode check its power, address, token and service log.

• **No model:** run `camera_app/fetch_models.py`; the UI exposes unavailable detector capabilities. Hash mismatches abort downloads.

• **Recording fails:** inspect `.encoder.log` under private offline media, verify `gst-inspect-1.0 x264enc`, stop recording before changing format.

• **App not starting:** verify Python dependencies and port availability; run in foreground to see errors. Do not run a second instance on the same port/data.

• **Low storage:** new writes are refused below 256 MB; recording stops near that threshold. Inputs require at least 1 GB free before import. No automatic media deletion is implemented.

• **Pi temperature:** ensure active cooling and adequate power. Preview/AI workload caps help but do not replace cooling.

Host MP4 requires a clean stop to finalize. The Pi backend also handles SIGTERM and finalizes its encoder through the capture owner before exit; deploy the generated Pi unit with `KillMode=mixed` so its FFmpeg child is not killed prematurely. SIGTERM and Ctrl-C trigger host cleanup. The generated host service uses `KillMode=mixed` and `TimeoutStopSec=30` so Python can finalize its encoder before systemd terminates remaining children. Abrupt power loss can damage an unfinished recording. The Pi baseline uses a different fragmented-MP4 output path; its existing resilience does not prove SD-card or power-loss safety.

Decoder measurement dependencies

The native receiver requires the GStreamer packages above. The optional quantitative monitor additionally uses the distribution Python bindings:

```
sudo apt install python3-gi gir1.2-gstreamer-1.0 python3-requests
/usr/bin/python3 airlink/monitor.py --help
```

A virtualenv may not expose system `gi`; use `/usr/bin/python3` for this monitor. Installation archives, fresh-board checks and the distinction between software deployment and a bootable image are described in Installation bundle.

Using Camera Studio

Sources

Simulator renders a moving test pattern with color bars, a grayscale ramp, a textured tracking target and a generated QR payload. Its visible SIMULATOR badge remains on the preview. Exposure, autofocus, denoise and HDR are disabled because they require physical hardware. Software transforms approximate appearance; they do not reproduce the Pi ISP.

Replay file uses a host-local input library. Click **+ Add file**, select MP4/AVI/MOV/MKV/WebM/JPEG/PNG, then **Apply source**. Files are copied to private UUID names. Video loops automatically; use Pause/Resume and the seek slider. The output FPS can differ from native file FPS; frames are repeated or skipped to match. Images have no timeline. Imports are limited to 512 MB per request and 16 megapixels per decoded frame. Codec availability depends on OpenCV's decoder build.

Raspberry Pi forwards requests to a configured local camera API. It never probes the other cameras on the LAN. A disconnected Pi stays unavailable until you choose another source; a simulator is never substituted for live camera imagery. Switching source while recording is rejected.

Camera and analysis

The Camera tab controls capture format, preview quality/FPS, recording bitrate, zoom and image appearance. On hardware, it also exposes sensor/ISP exposure, white balance, focus and processing HDR. Actual sensor metadata is shown separately from requested settings. Format changes interrupt capture briefly. Some controls depend on sensor and Pi capabilities.

Open **Profiles & saved presets** for Pi 5 Balanced, Pi 3 A+ Candidate and Offline Development. The Pi 3 profile is a proposed low-load starting point, not a performance guarantee. Save a named preset on Ubuntu, export JSON for version control, or import version-1 Camera Studio preset JSON. Applying a full preset during recording is blocked. Physical settings are skipped when applying it to offline sources, and the UI reports that fact.

AI & CV modes:

• **Objects:** CPU FP32 MobileNet-SSD; 20 PASCAL VOC classes, confidence threshold adjustable. Not general open-vocabulary recognition.

• **Faces:** OpenCV Haar face locations, no identity recognition.

• **QR:** decoded text, never automatically navigated or executed. Includes a single-code fallback when the multi-code detector misses a small code.

• **Motion:** background subtraction with ten-cycle warmup; assumes a stationary camera.

• **Region tracking:** select a textured region by dragging. A local template matcher follows translations and declares loss below confidence 0.55; large rotation or scale changes are outside its design.

Analysis works on reduced 640 x 360 frames independently of capture. Increasing the interval reduces CPU use. Overlays are drawn in the browser; photos/video remain clean. Analysis age is shown and stale boxes are hidden. The histogram, clipped/dark percentages and detail score describe the reduced image. Detail score varies with noise and scene texture; it is not a calibrated focus measurement.

Photos, video and lab

Snapshot saves a JPEG and JSON sidecar at the current output resolution. **12 MP photo** is hardware-only, pauses preview briefly and is blocked during recording. **Record video** writes H.264 MP4 and JSONL metadata. Stop it before downloading or changing capture geometry/bitrate. Media lets you preview finished MP4s and download files. Files belong to the selected source: Pi storage in Pi mode, Ubuntu storage in offline modes.

Keyboard shortcuts: **S** snapshot, **R** toggle recording, **Escape** cancel ROI selection. Shortcuts are suppressed while editing form fields.

The **Lab** tab analyzes a saved host input regardless of the active preview source. Enable the desired AI/CV modes, select an input, frame stride (1..120), and maximum analyzed frames (1..1000), then start. Only one job runs at a time. Reports contain per-frame output, p50/p95 processing time, class counts, QR payloads and image quality metrics. Download JSON for detail or CSV for tabular analysis. Report timing measures this Ubuntu host; it does not estimate Pi speed or labeled-dataset accuracy.

Interpreting status

Capture FPS and preview FPS are independent. An idle source generates JPEGs at at most 2 FPS until a viewer connects. The Pi may cap preview/analysis at high temperature. Host frame age and HTTP round-trip time are not optical glass-to-glass latency. Offline sources show no fabricated sensor temperature, gain or focus metadata.

The browser UI, diagnostics and all project documentation are in English. No cloud service receives camera frames.

Link and native video

Select the physical Raspberry Pi source, then open **Link**. Start the native receiver command shown there on Ubuntu. Set its explicit receiver IPv4 address, port and MTU, then click **Start RTP**. Confirm the active state and increasing frame/packet counters. Click **Stop RTP** to release the sender. Simulator and Replay reject physical RTP transmission.

The same encoder can serve recording and RTP simultaneously. Stopping one keeps the other active. Format, rate, HDR, rotation, bitrate, full-sensor capture, restart and source selection stay locked while encoding. Exposure/white balance and AI options can still be adjusted. A camera service restart does not automatically resume network transmission.

The **Latency target** link opens a changing visual timer for a later optical measurement. Aim the physical camera at the host display and use an independent high-speed reference to compare source and received views. The browser timer alone does not measure latency. Software decoder measurements are documented separately and exclude sensor/encoding/display delay.



Optical test reference; this generated timer is not a measured latency result.

Architecture and API

Execution boundaries

```
Browser / localhost host UI  ->  explicit source manager
  ->  Local simulator or replay / Ubuntu vision
  ->  Pi API / Picamera2 / ISP / Pi vision

Air link target: one camera -> H.264 -> RTP -> WFB
  ->  ground RTP decoder -> native display
Camera encoder fanout validated on Ethernet; RF pending
```

`host.py` owns the browser application and proxies a selected endpoint. `host_services.py` manages source selection, presets, profiles and the offline API. `camera_api.py` is the common authenticated contract. `backend.py` lazily imports the Pi engine, so host imports do not need camera drivers.

`configuration.py` supplies shared validation. `engine.py` owns physical camera commands, capture, encoding, analysis and watchdog recovery. `offline.py` owns software capture/replay and uses the same API. Its capture and vision loops are independent; latest frames replace older analysis work. `vision.py` implements CPU algorithms and normalized boxes. `recording.py` uses a bounded four-frame host encoder queue, with drop/error counters. `benchmark.py` processes a bounded file job with separate Vision state.

The physical engine remains a single camera owner. Do not start a second Picamera2/rpicam/OpenHD capture process alongside it. The backend fans one encoder output into recording and RTP through `encoded_outputs.py`. `rtp.py` packetizes existing H.264 bytes and sends through a bounded nonblocking socket; it never acquires the sensor. The standalone transport tools also accept synthetic or already encoded video.

API contract

Host at localhost:8765; Pi camera API at private-address:8081. POSTs to the host need `X-Camera-Client: 1`. Calls directly to a camera API need `X-Camera-Token: <private token>`. Browser requests never receive the token. Camera JSON payload limit: 16 KB. Host uploads: 512 MB.

Endpoint	Method	Purpose
`/api/health`	GET	Host version and selected source, no hardware probe
`/api/host`	GET	Mode, input library and profiles
`/api/source`	POST	`{mode, input_id?, camera_url?}`; mode simulator/replay/pi
`/api/inputs`	GET / POST	List inputs / multipart field `file`
`/api/playback`	POST	`{paused?, seek_s?}` for replay
`/api/status`	GET	Settings, metadata, capabilities, analysis, recording, rates, events
`/stream.mjpg`	GET	Multipart JPEG preview
`/api/frame.jpg`	GET	Latest preview JPEG
`/api/settings`	POST	Validated partial camera settings
`/api/transport`	POST	`{enabled:true,host,port?,mtu?}` or `{enabled:false}`; real Pi only

Endpoint	Method	Purpose
`/api/clock`	GET	Unix/monotonic nanoseconds for software clock calibration
`/latency-target`	GET	Host browser optical-test timer, no automatic latency result
`/api/ai`	POST	objects/faces/qr/motion booleans, confidence, interval_ms
`/api/track`	POST	`{roi:{x,y,w,h}}` normalized coordinates; null clears
`/api/action/<name>`	POST	snapshot, snapshot_full, record_start, record_stop, restart, autofocus
`/api/media`	GET	Current source's media list
`/api/media/<name>`	GET	Download finished media/sidecars
`/api/media/view/<name>`	GET	Host inline media, Range support
`/api/presets`	GET / POST	List / save current `{name}` or import `{preset}`
`/api/presets/<id>`	GET	Export portable JSON
`/api/presets/<id>/apply`	POST	Apply validated preset
`/api/profiles/<id>/apply`	POST	Apply built-in profile
`/api/benchmarks`	GET / POST	Reports/progress / start input analysis
`/api/reports/<id>.json or .csv`	GET	Download a completed host report

Host-only endpoints are not implemented on the Pi. Examples:

```
curl http://127.0.0.1:8765/api/status
curl -H 'X-Camera-Client: 1' -H 'Content-Type: application/json' \
  -d '{"mode":"simulator"}' http://127.0.0.1:8765/api/source
curl -H 'X-Camera-Client: 1' -H 'Content-Type: application/json' \
  -d '{"qr":true,"interval_ms":350}' http://127.0.0.1:8765/api/ai
```

Validation rejects unknown keys, nonfinite values, wrong types and invalid ranges. Complete resolution pairs are checked against 640x360, 1280x720 and 1920x1080. Host presets use `format: camera-studio-preset`, `version: 1`, name, camera object and optional analysis object. Applying a preset filters unsupported physical settings in offline mode, then applies camera and AI values; camera settings are rolled back if AI application fails.

Encoded transport state

`/api/status.transport` reports support, active state, destination options, codec/payload type, sender counters and `auto_resume:false`. Its counters distinguish encoded frames, keyframes, successful datagrams/bytes, local socket drops and malformed encoder input. They do not establish receiver delivery. `/api/status.encoding` reports one encoder generation and current consumers (`rtp`, `recording`), so lifecycle tests can detect unintended encoder replacement.

Starting the first consumer starts encoding; stopping the last releases it. Camera service shutdown finalizes the recording and closes RTP. Stream destinations are never resumed automatically after a restart. The host Link tab proxies explicit start/stop requests; Simulator/Replay reject physical transport. The native receiver uses standard RTP caps and ignores the optional private timestamp extension. Software clock validity is checked separately from decoded-frame counts and packet loss.

Private storage

- `camera_app/runtime.json`: shared token + Pi URL, mode 0600.
- `host-data/session.json`: selected source; no automatic network discovery.
- `host-data/inputs/`: UUID filenames + probed index; imported videos remain private.
- `host-data/presets.json`: named host presets.
- `host-data/offline/`: software settings and analysis; `media/` contains JPEG/MP4/JSON/JSONL and encoder logs.
- `host-data/reports/`: analysis JSON/CSV, potentially including input names and QR contents.
- Pi `~/camera_app/data/`: physical settings, media and metadata.

JSON state writes use unique temporary files, fsync and atomic replacement. There is no automatic retention/deletion policy. Keep reports private when inputs or QR content are private. A single host process owns each data directory; multiple independent writers are not supported.

Trust and deployment limits

The host listens only on loopback. Its write header limits ordinary cross-site form writes but is not a multiuser authentication system. Camera URLs are restricted to private/loopback IPs or `.local` names without credentials or paths. The Pi listens on its network interfaces and uses token authentication over plain HTTP. Use a trusted development LAN or an encrypted private tunnel; do not expose port 8081 publicly. TLS, Internet auth, discovery and role-based access are outside this version.

A source change closes the old stream generation and reconnects the preview. Failures remain attributed to the selected source. No GigE Vision discovery, address reassignment or NIC reconfiguration is performed.

Airborne video transport

Decision and current implementation

Use **H.264 over RTP/UDP** as the video contract. Keep browser MJPEG for local configuration. Prepare a dedicated **WFB-ng** air/ground radio link once compatible USB adapters are available. This is an architecture choice for later hardware evaluation, not an installed radio stack or a range guarantee.

The repository implements real Pi camera H.264/RTP transmission, synthetic transmission, a native ground receiver, measured decoder monitoring, conservative profiles and a read-only adapter/driver inventory. Ethernet validation exercises the Pi encoder and actual host decoder. Radio hardware remains untested.

```
Browser / localhost host UI  ->  explicit source manager
  ->  Local simulator or replay / Ubuntu vision
  ->  Pi API / Picamera2 / ISP / Pi vision

Air link target: one camera -> H.264 -> RTP -> WFB
  ->  ground RTP decoder -> native display
Camera encoder fanout validated on Ethernet; RF pending
```

The Pi uses one camera owner and one H.264 encoder. `CameraOutputs` fans its encoded access units out to a nonblocking RTP sender and optional MP4 recorder. Starting/stopping either consumer preserves the other. Geometry, frame rate, HDR, rotation, bitrate, full-resolution capture and source changes are locked while encoding. Recording attached to an existing stream waits for the next IDR (up to about one second).

Real camera over Ethernet

Start the receiver on the Ubuntu host:

```
python3 airlink/video.py receive --host YOUR_HOST_IPV4 --port 5600 --profile
pi5-candidate
```

In Camera Studio select **Raspberry Pi**, open **Link**, enter that same receiver IPv4/port and click **Start RTP**. The browser remains an MJPEG configuration preview; the native receiver decodes H.264. The suggested receiver address comes from the host route to the explicitly selected Pi, without network scanning. Stop RTP before deploying or changing the encoding profile.

The sender accepts explicit private/loopback IPv4 unicast addresses, ports 1024-65535 and RTP MTUs 256-1400 bytes. It does not discover cameras or alter network interfaces. Restarting the camera service leaves RTP **off**; explicitly enable it again after recovery. Plain UDP provides no encryption, retransmission or delivery guarantee.

`RtpSender` implements Annex-B single-NAL and FU-A fragmentation according to RFC 6184. It repeats the encoder's SPS/PPS and one-second IDRs. Its bounded kernel socket queue drops rather than accumulating unlimited work. Status exposes sent bytes/packets/frames, local drops and errors. The bitrate counter includes RTP headers, but excludes UDP/IP and radio overhead. Native RTP is also accepted by the standard GStreamer depayloader in the automated interoperability test.

For quantitative headless monitoring, use the system Python with GI/GStreamer:

```
/usr/bin/python3 airlink/monitor.py --bind YOUR_HOST_IPV4 --port 5600 \
  --seconds 60 --output artifacts/receiver.json
python3 airlink/budget.py --help
python3 airlink/impair.py --help
```

Both receivers separate UDP reads from decoding through a leaky ingress queue bounded at 256 packets / 307200 bytes (no queue-time limit across idle gaps) and explicitly request a receive socket buffer. OS limits still apply; no system-wide buffer tuning is performed. The monitor stores aggregate measurements, never pixels. A private RTP extension (0xCD01, eight-byte Unix send timestamp) supports **encoder-output-to-decoded-frame** latency measurements, with before/after HTTP clock calibration and uncertainty. This excludes exposure, ISP, encoding and display. Standard receivers

ignore the extension. A browser timer at `/latency-target` is provided for a later physical optical test; its precision is bounded by display refresh and browser scheduling. No optical result is inferred from software timestamps.

The bounded UDP relay can inject a repeatable blackout/loss into this stream without changing the NIC, firewall or another camera. See real transport validation for measured coexistence, interruption and recovery results.

Local bench

Install the host GStreamer packages listed in README. In separate terminals:

```
python3 airlink/video.py receive --profile bench
python3 airlink/video.py send-test --profile bench
```

Both default to 127.0.0.1:5600. Use `--headless` on receive without a display, or `--print-only` to inspect commands. For a LAN receiver, bind `--host` to its own private interface IP, and set sender `--host` to that address. No firewall settings are changed automatically. Stop with Ctrl-C.

An existing encoder can pipe an Annex-B H.264 byte stream into:

```
python3 airlink/video.py send-h264-stdin --host 127.0.0.1 --port 5600
```

The encoder must supply repeated IDRs and SPS/PPS, appropriate bitrate and dimensions. `send-h264-stdin` packetizes only; the profile does not reconfigure an upstream encoder. Do not launch `rpicam-vid` to supply it while Camera Studio owns the sensor.

Run the repeatable headless bench:

```
python3 -m unittest discover -s tests -p test_airlink.py -v
python3 airlink/diagnose.py
```

Observed loopback result: 29 decoded 640x360 frames from a 60-frame finite sender, with receive stopped after its configured buffer count. Receiver start/stop means this is a pipeline smoke test, not packet-loss estimation. Full sender/receiver E2E latency was not measured.

Packet and queue contract

Parameter	Initial value	Reason / limit
Codec	H.264	Native ground decode; camera encoding remains platform dependent
RTP payload / clock	96 / 90000 Hz	Explicit caps at receiver
UDP port	5600	Local contract; configure WFB endpoints to match
RTP MTU	1200 bytes	Candidate to avoid large IP packets; tune against actual radio MTU
IDR interval	One second in Pi and synthetic encoders	Candidate recovery tradeoff; measure after packet loss
SPS/PPS	Repeated encoder headers / `config-interval=-1`	Pi / synthetic sender respectively
Jitter buffer	30 ms bench/Pi5, 40 ms Pi3 candidate	Bounded waiting, late packets dropped
Synthetic encoder	x264 ultrafast / zerolatency / no B frames	Host bench, not a Pi encoder selection
Bench	640x360, 15 FPS, 1.5 Mbit/s	Low-load starting point
Pi 5 candidate	1280x720, 30 FPS, 4 Mbit/s	Must be remeasured with radio + inference

SPS/PPS behavior is documented by GStreamer rtph264pay. The jitterbuffer reorders packets, waits only up to its latency setting and adds that latency to the pipeline. A lower value is not automatically better on a variable radio channel.

Payload budget

At 4 Mbit/s video, MTU 1200, an illustrative 12/8 FEC ratio and 64 kbit/s control reserve, `airlink/budget.py` estimates approximately **6.34 Mbit/s** of payload capacity before unmodeled radio/MAC/security/FEC-block headers, bitrate bursts and fading margin. The FEC ratio is a calculation input, not an installed or validated radio configuration. Enter measured usable payload capacity, never the advertised PHY link rate.

Radio software and drivers

WFB-ng maps video packets onto raw WiFi frames with forward error correction and authenticated encryption. It requires compatible injection/reception adapters and suitable drivers; ordinary managed WiFi is not equivalent. Consult the upstream WFB-ng repository and setup guide for the exact current hardware and build instructions.

The upstream project lists patched RTL8812AU and RTL8812EU driver paths. Match the **exact USB chipset, driver branch, Linux kernel, architecture and adapter power/thermal design** before installing a driver. A brand name or `lsusb` entry alone does not prove compatibility. Upstream driver projects: rtl8812au and rtl8812eu. No third-party kernel modules, blacklist rules, monitor-mode changes or TX-power overrides are installed by this project.

The current Ubuntu inventory has only Ethernet and loopback interfaces, with no wireless network interface. Its required GStreamer elements are available. `iw` and `ethtool` were not present in the inspected host environment. Once adapters arrive, install those OS tools as needed, rerun inventory, and validate WFB injection/RX against the exact kernel before changing the operational network. Keep

SSH management on Ethernet during initial radio work.

OpenHD is an alternative integrated video/telemetry stack. Evaluate it as a separate image or documented external-video integration. Its camera capture service must not contend with this application's camera owner. Do not mix arbitrary OpenHD kernels/drivers with WFB packages without a compatibility plan.

Control and AI metadata should use a separately budgeted private IP tunnel or dedicated telemetry link. Do not stream the high-bandwidth HTTP MJPEG preview through that control tunnel. Wire encryption comes from a configured radio/tunnel stack, not from plain RTP/UDP or the camera's API token.

Hardware acceptance sequence

1. Inventory both USB adapters, board OS/kernel and drivers; keep management Ethernet working.

2. Repeat the synthetic RTP bench through WFB with a physical link before attaching a camera stream.

3. Use the real Pi encoded stream with AI off. Verify stable decode, packet/FEC counters, CPU, temperature and power.

4. Introduce controlled obstruction/interruption; measure freeze duration and recovery at successive IDRs.

5. Add low-rate AI metadata, then onboard recording, checking that neither causes queue growth.

6. Measure optical latency using a timer/LED visible to the camera and the ground display. Report p50/p95 and methodology.

7. Progress from a bench to restrained outdoor tests with measured link margins, safe power and correct local RF configuration.

No range, control-link availability, flight readiness or driver compatibility is asserted by a localhost test. Automatic bitrate adaptation, retransmission, WebRTC, MAVLink overlays and flight-controller integration are future work.

Validation report

Scope of evidence

This document separates the **historical Pi 5 baseline** from the **current offline extension**, validated on Ubuntu on 2026-11-03. The Pi was unplugged during the initial offline extension. It subsequently returned and the current version was redeployed and exercised on the real Pi 5. See the 2026-11-03 hardware regression, including the retained startup-rate assertion failure and separate steady-state measurements. Historical results below remain evidence for their original baseline. Private raw artifacts and real camera images are intentionally excluded from the public repository.

Validation used the real Raspberry Pi 5 (8 GB) and its CSI IMX708 camera. The Ubuntu application at `http://127.0.0.1:8765/` was the control endpoint. No GigE Vision camera was accessed or configured.

Real camera checks

All **18 hardware checks passed** in the final functional run:

• Reject unknown settings, unsupported sizes and invalid confidence thresholds.

• Manual exposure of 5000 us produced 4991 us in sensor metadata; gain 2.0 was reported as 2.0.

• Manual white balance gains 1.6 / 1.2 appeared in ISP metadata.

• Brightness, contrast, saturation, sharpening, exposure compensation and denoise controls were accepted.

• Zoom 2 changed the capture crop from `[768,432,3072,1728]` to `[1536,864,1536,864]`.

• 640 x 360 at 15 FPS, 1920 x 1080 at 30 FPS and 1280 x 720 at 30 FPS captured correctly sized JPEGs.

• 180-degree rotation and Pi 5 single-exposure HDR reconfiguration resumed capture.

• Autofocus was exercised; a separate metadata check observed scanning followed by focused state, with lens position moving from 0 to 12 diopters.

• Standard snapshots and JSON metadata downloaded successfully.

• A full-resolution photo decoded as **4608 x 2592**; the 720p preview recovered afterward.

• A five-second MP4 decoded successfully into **155 frames** at 1280 x 720.

• Resolution/bitrate/full-photo changes and duplicate recording starts were rejected while recording.

• All four analysis modes ran together on the live image without stopping 30 FPS capture.

• Live-region tracking reported approximately 0.997 match confidence on a stationary region.

• The camera restart operation resumed capture.

Details: `artifacts/hardware-results.json`. JPEGs and a decoded test recording are also saved in `artifacts/`.

Known-input AI/CV checks

All **5 known-input tests passed on the Pi CPU**:

1. MobileNet-SSD detected a person in the model author's sample image.
2. OpenCV's face cascade found a face in the OpenCV Lena sample.
3. QR decoding recovered the generated payload `CameraStudio-QR-test`.
4. Background subtraction found a newly introduced moving region after warmup.
5. Template tracking followed a known translation and declared loss when the target disappeared.

These checks establish working model execution and algorithm behavior, not field accuracy across arbitrary lighting, object sizes or viewpoints.

Details: `artifacts/vision-results.json`. Sample sources: the MobileNet-SSD repository's `images/000001.jpg` and OpenCV's `samples/data/lena.jpg`. These are test fixtures, not camera media.

Ten-minute live soak

A continuous MJPEG client ran with CPU object detection every 350 ms. The first minute also recorded H.264 MP4 on the Pi.

Observation	Result
Duration	602.5 seconds
Preview JPEG frames received	7310
Stream errors	None
Capture FPS range	28.9-30.1
Highest sampled temperature	73.2 C
Highest sampled frame age on Pi	46 ms
Preview data received	534,434,043 bytes

Frame age measures freshness inside the Pi service. It does not include network, browser buffering or display latency. Preview target was 15 FPS; actual delivery was approximately 12 FPS under this workload. Preview and capture rates are reported independently in the application.

The initial implementation reached roughly 83 C and a firmware temperature limit under recording load. After reducing redundant frame copies/conversions, the continuous soak above remained below 74 C. Thermal workload caps and firmware flags are exposed in the application. Historical firmware limit flags can remain set even after the current condition clears.

The soak preceded the final additions for 12 MP photos, recording bitrate selection and stricter input validation. The unchanged capture/analysis pipeline was soaked; those additions were subsequently covered by the final hardware checks.

Details: `artifacts/soak-results.json`.

Recovery, persistence and access

• Forced termination of the Pi backend process: camera recovered in **4.77 seconds**.

• Camera and analysis settings survived that recovery.

• Software reboot: the camera API returned automatically after approximately 29 seconds,

before any SSH login. Capture returned to 30 FPS and AI settings were preserved.

• Host writes without the application header were rejected with HTTP 403.

• Pi API calls without the shared token were rejected with HTTP 401.

• Ten invalid-input cases were rejected with HTTP 400, including NaN, wrong types,

out-of-range settings and malformed tracking regions.

• The Pi user service is enabled with `Linger=yes`; the host service is enabled

for the Ubuntu user session.

• Browser checks exercised settings, snapshots, the Media gallery, downloading a

photo to Ubuntu, and selecting a tracking region by dragging on the live view.

• Application source strings were checked for remaining Cyrillic text.

Details: `artifacts/recovery-results.json`, `artifacts/invalid-input-results.json` and `artifacts/reboot-results.json`.

Verification limits

• Optical end-to-end latency has not been measured.

• Physical power removal, power quality and SD-card failure have not been tested.

• Low-disk protection is implemented but the real card was not filled to test it.

• No long-duration 1080p recording or field-accuracy benchmark was performed.

• Pi 3 A+ portability and performance remain future work.

• HDR and focus commands were functionally exercised; controlled optical comparisons and sensor HDR tuning remain future work.

Current Ubuntu offline extension (2026-11-03)

The initial offline extension passed **12 tests** (the RTP extension raises the current suite to **19**):

Check	Observed result
Synthetic capture / JPEG / multipart preview	Running; valid JPEG frames
API authentication / host write header	Missing token 401, missing application header 403
Hardware-only controls offline	Exposure and full-sensor photo rejected, 400
Resolution validation and QR fallback	Invalid size pair rejected; generated testcard payload decoded at 640x360
Preset save/export/apply and Pi3 candidate profile	Passed with explicit unsupported-setting filtering
Local recording	H.264 MP4 stopped cleanly; every submitted/encoded frame decoded
Recording guards and HTTP Range media view	Source change/download blocked during recording; 206 partial response afterward
Input upload, replay, pause/seek and bounded batch report	Synthetic PNG and generated MP4; JSON/CSV downloads passed
Offline source isolation	Requests stayed on localhost despite an unreachable configured Pi address
Numeric / type / URL boundary validation	Invalid values rejected, including nonfinite values and nonlocal endpoints
Motion and tracking known synthetic transforms	Motion detected; translation followed and target loss declared
Real H.264/RTP localhost transport	29 decoded 640x360 frames; encode, UDP and decode pipeline all executed

Five additional known-input checks passed on the Ubuntu CPU: object detection, face detection, QR decoding, background motion and tracking/loss. These use the historical optional upstream sample photographs and generated patterns; the photographs are not redistributed. This does not establish general detection accuracy.

The recording test exposed malformed fragmented-MP4 output from this Ubuntu GStreamer stack: a weak first-frame-only check could pass even though subsequent NAL reads failed. The host recorder now uses finalized fast-start MP4 with I420 input, and the regression check decodes **all** encoded frames. This change is host-only; the Pi's FFmpeg output path remains separate. Host recordings require a clean stop and do not claim power-loss resilience.

Browser review verified Simulator labeling, generated QR overlay/text, unavailable hardware controls and the English source/AI interface. The public screenshot is synthetic. Python compilation, JavaScript syntax and shell syntax checks passed. GitHub Actions is configured but remote execution is not claimed until its actual result is observed.

Read-only radio inventory: Ubuntu x86_64, Ethernet `e1000e` and loopback only; no wireless network interface. Required GStreamer encoder/RTP/decode elements are present. No WiFi driver/module, monitor mode, channel, power or other camera configuration was changed.

Offline soak and service shutdown

A separate **60.69-second** synthetic-source soak ran all four AI/CV modes, preview JPEG generation and an initial recording on Ubuntu. Minimum sampled capture rate after warmup was **30.0 FPS**, maximum sampled frame age **26 ms**, and **867 preview frames** were generated. The recording contained **451 decoded frames**, with **zero encoder queue drops**. These figures describe this host, not Raspberry Pi or RF behavior.

A real `systemctl --user restart camera-studio.service` during a synthetic recording initially exposed process-group termination interrupting MP4 finalization. The installed/generated host unit now uses `KillMode=mixed` and a 30-second stop timeout, while Python handles SIGTERM cleanup. The repeated restart test produced a finalized MP4 with **53 decoded frames**, and the source returned running at 30 FPS with no active recording. Abrupt power loss remains untested.

Reproduce

```
python3 -m unittest discover -s tests -p 'test_*.py' -v
python3 tools/fetch_test_fixtures.py  # optional external samples
python3 tests/vision_check.py       # optional known-image suite
python3 airlink/diagnose.py
python3 tools/offline_soak.py --seconds 60 --output artifacts/offline-soak.json
```

Hardware scripts `tests/hardware_check.py` and `tests/soak_check.py` change settings and create media. Select the real Pi deliberately before running them. They are not part of offline unittest discovery.

Remaining unverified behavior

Physical cold boot and power loss, Pi3 encoder/driver/RAM suitability, actual RF adapter support, RF packet loss/FEC/recovery, optical latency, range, field accuracy, storage-exhaustion behavior and long-term 1080p operation still need physical tests. A localhost transport test does not establish any of those results.

Real Pi RTP extension (release 0.3)

See the dedicated transport report for single-encoder recording fanout, real Ethernet decode, timestamp latency methodology, controlled blackout and service-restart recovery. The current host suite has 19 passing tests. Software timestamp latency excludes exposure, ISP, encoding and display; it is not a replacement for an optical test.

The RTP extension also retains initial buffer/queue failures and the final direct regression's 969.520 ms maximum decoded-frame gap. Passing the current functional acceptance does not establish hard real-time delivery; see the dedicated report for complete results and clock validity.

Raspberry Pi 5 regression - 2026-11-03

Scope and equipment

The Raspberry Pi returned to the network and the current application was deployed over its previously verified SSH identity. The CSI stack identified **Sony IMX708, 4608 x 2592**, on **Raspberry Pi 5 Model B Rev 1.0**. The separate GigE Vision camera was not accessed or configured.

Environment: Debian 13.7 (trixie), kernel `6.18.50+rpt-rpi-2712`, Python 3.13.5, Picamera2 0.3.37, OpenCV 4.10.0. The long run initially exercised the published `a616f3f` Pi backend. Subsequent lifecycle changes, host control refresh and test-safety fixes accompany this report; their separate retests are identified below. A private pre-deployment source/configuration backup was retained on the Pi.

Functional regression

All **18 hardware checks passed**. These exercised exposure/gain metadata, manual white balance, ISP appearance controls, zoom crop, three output sizes, FPS, rotation, processing HDR, autofocus commands, JPEG/JSON snapshots, a full-resolution photo, H.264 recording, simultaneous analysis modes, tracking and camera restart.

• Manual 5000 us exposure was reported as 4991 us; requested gain 2.0 was reported as 2.0.

• A full-sensor photo decoded at 4608 x 2592, followed by preview recovery.

• A five-second 720p recording decoded into 155 frames.

• All AI/CV modes ran together while capture remained at 30 FPS.

• Five known-input CV checks passed directly on the Pi: objects, face location, QR, background motion, and tracker translation/loss.

These verify working controls and algorithms, not calibrated optical HDR/focus quality or field detection accuracy.

Continuous capture and recording

The primary run used object detection every 350 ms, a continuous MJPEG receiver, and approximately one minute of initial H.264 recording. Additional browser clients and a short independent known-input CV check were present during the run; it was not an isolated microbenchmark.

Measurement	Result
Run duration	603.1 seconds
Preview frames received	7386
Preview bytes received	422,081,566
Stream errors	None
Capture FPS across every sample	22.3-30.1
Capture FPS after the first 10 seconds	30.0-30.1
Highest sampled Pi temperature	67.8 C
Highest sampled frame age	37 ms

Measurement	Result
Nonzero current firmware limit flags	0 samples
Minute recording decoded frames	1811
Minute recording reported FPS	30.032

The original soak invocation **failed its strict minimum-FPS assertion** because its very first sample, at elapsed 0.0 s immediately after service recovery/encoder startup, reported 22.3 FPS. This was the only sample below 28 FPS; all 118 samples at elapsed 10 s and later measured 30.0-30.1 FPS. No stream error, stale-frame failure or AI error occurred. The raw startup observation is preserved, not discarded or relabeled as a successful strict all-sample test.

The script now explicitly reports both overall and steady-state minima, with a documented 10-second warmup for the steady-rate acceptance check. Fresh-frame, running-state and AI-error checks still apply throughout startup. `--duration` supports a shorter follow-up without changing the default ten-minute run. Warmup does not justify a later low-FPS interval.

Short confirmation after test changes

The guarded functional script passed all 18 checks again, then the revised soak passed a separate **60.3-second** repeat. Overall capture range was 29.9-30.2 FPS; after the explicit 10-second warmup it was 29.9-30.1 FPS. It received 782 preview frames without a stream error. Highest sampled temperature was 68.8 C and frame age 34 ms. This short repeat confirms the revised test behavior; it does not replace the raw ten-minute observations above.

Recovery and host controls

Switching Simulator to Pi and back passed. Forced termination of the Pi backend recovered in **5.04 seconds** with camera and AI settings preserved. Boot lingering was already enabled; this run did not physically remove power.

Browser regression exposed host controls retaining their old disabled state after a source change. The UI now refreshes controls and the selector when the source generation changes, including changes made in another client. The autofocus button is explicitly re-enabled for supported hardware. Browser checks confirmed exposure/focus enabled on Pi and disabled after returning to Simulator. Recording format locks remain applied after initial control synchronization.

Hardware scripts now refuse Simulator, unavailable hardware and an existing recording. The functional test restores the original analysis settings as well as camera settings.

Recording integrity during an ordinary service restart

The initial ordinary service-restart check found a damaged MP4 tail: 76 frames remained readable, but the decoder reported invalid NAL units and a partial file. A readable first frame or a plausible frame count was therefore insufficient evidence of recording integrity.

The Pi entry point now handles SIGTERM and calls `CameraEngine.close()`. The capture owner finalizes its encoder and metadata, closes the camera, and finishes before process exit. Preview iterators wake during shutdown, and new commands are rejected while stopping. The generated/example Pi systemd unit uses `KillMode=mixed` and a 30-second stop timeout so FFmpeg is not terminated before the camera owner finishes.

A repeated 3.5-second recording/restart test decoded **111 frames with zero decoder errors** using `ffmpeg -v error -xerror -err_detect explode`. Camera recovery took **4.14 seconds**. The decode test uses a precise 1/90000 output time base with passthrough frame timing: this avoids null-output timestamp rounding warnings on valid variable-rate MP4 while retaining strict NAL/truncation failure checks. The original broken sample was still rejected with this configuration. This is a clean service-stop result, not evidence of power-loss safety. `tests/restart_recording_check.py` reproduces

the test, requires an explicit SSH hostname matching the selected camera URL, checks that no recording is already active, and verifies persisted settings.

Regression after the Pi shutdown fix

The reproducible restart test passed with **112 strictly decoded frames**, zero decoder errors, **4.83 seconds** recovery and persisted camera/AI settings. The complete hardware script then passed all 18 checks again.

A subsequent **60.5-second** soak on the updated backend passed: overall capture 30.0-30.4 FPS, **30.0-30.0 FPS** after its first 10 seconds, 746 preview frames, no stream errors, maximum temperature 67.8 C and maximum frame age 40 ms. The ten-minute run above preceded the lifecycle fix; this shorter run and restart test cover that subsequent change.

Reproduction and limits

```
python3 tests/hardware_check.py
python3 tests/restart_recording_check.py --pi-target youruser@192.168.1.20
python3 tests/soak_check.py                  # 600 seconds
python3 tests/soak_check.py --duration 60    # shorter repeat
```

Select a running real Pi deliberately, stop existing recording, and choose at least 25 FPS for the soak. These commands change settings and create local media. Private logs, real images, recordings, runtime tokens and host keys are excluded from the public repository.

A 30-60 minute thermal run, physical cold boot, power-loss behavior, Pi 3 A+, actual camera-to-RTP integration, radio performance and optical end-to-end latency remain unverified. The current browser preview remains a LAN configuration stream.

Real camera transport validation - 3 November 2026

Scope and platform

Release 0.3 adds one-encoder H.264 recording/RTP fanout to the real Picamera2 backend. Testing used Raspberry Pi 5 (8 GB), CSI IMX708, Ethernet and the Ubuntu host's standard GStreamer H.264 decoder. No GigE Vision camera, WiFi interface, radio driver, bootloader or EEPROM was modified. Private raw logs and camera media remain outside the public repository.

The Pi ran Debian 13.7, kernel 6.18.50+rpt-rpi-2712, Python 3.13.5, Picamera2 0.3.37 and OpenCV 4.10.0. The Ubuntu decoder used GStreamer 1.20.3. `tools/environment.py` exports nonsecret package/source hashes for local reproduction.

Functional transport checks

Check	Measured result
Initial real RTP decode	115 decoded 1280x720 frames; 30.169 FPS; no loss
RTP plus recording	594 decoded frames over approximately 20 seconds; 30.048 FPS; no decode errors or packet loss
Shared encoder	Encoder generation unchanged when attaching/stopping recording; stopping RTP preserved a concurrent recording
MP4 coexistence	Six-second recording decoded strictly with FFmpeg; complete file, no decode errors
Controlled interruption	Bounded UDP relay dropped packets during a two-second blackout; decoder recovered without restart
Blackout observation	567 frames in 24.316 seconds, maximum decoded-frame gap 2621.387 ms; receiver reported 911 lost packets, matching relay drops
Service restart	Recording finalized (161 decoded frames, zero errors); service returned in 4.15 seconds
RTP after service restart	Sender stayed off until explicitly enabled; existing receiver accepted the new RTP session; maximum decode gap 4926.603 ms
Provisioning check	Read-only Pi OS/module/FFmpeg checks passed on the actual board
Software reboot	Boot identity changed; API returned in 32.23 seconds, before post-boot SSH; capture 30 FPS, settings/AI preserved, RTP off
Final functional regression	18 camera checks and six transport lifecycle/guard checks passed
Corrected-run recording	First-minute MP4 decoded to 1802 frames on Ubuntu, EOS reached, no errors/warnings; GStreamer avdec_h264 with max-errors=0 and output-corrupt=false

The relay test changes only its own UDP forwarding. It is not an RF fading/FEC test. A successful recovered stream can still contain a visible freeze. Maximum decoded-frame gap is reported separately from average FPS and timestamp latency.

Initial extended run: retained failure

The first long run was deliberately stopped after **1147.368 seconds** when a stability failure appeared. It decoded 32419 frames at 28.294 FPS, but reported **487 lost RTP packets** and a **17041.321 ms** maximum decode gap. Pi capture/encoding continued at approximately 30 FPS with no local sender drops. Linux reported 457 receive-buffer drops on this receiver socket; this is a host socket counter, not an RF measurement.

The original receive path used the default 212992-byte kernel buffer and ran decoder/measurement callbacks directly downstream of the network reader. The revised monitor and native receiver request a bounded socket buffer and add a bounded, leaky ingress queue (256 packets or 307200 bytes; whichever limit is reached first). This separates network reads from downstream stalls and discards stale backlog. No global sysctl, NIC, firewall or driver settings were changed. GStreamer documents the socket buffer setting and OS limits.

This observation motivates the receiver change; it does not prove that every missing packet had the same cause. The failed run remains separate from subsequent validation. Average FPS and median software latency alone did not establish stable delivery.

Corrected 30-minute run: passed

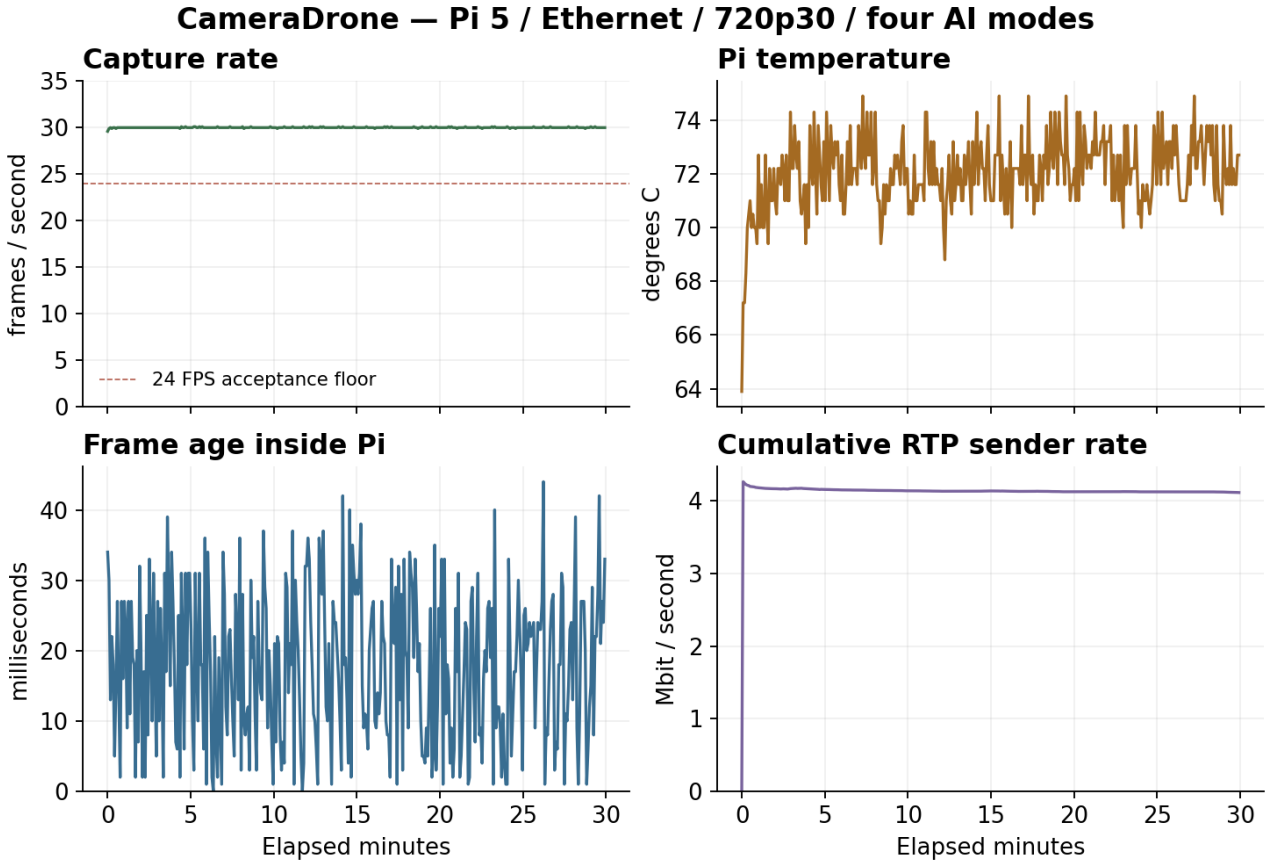
The fresh run completed with four CPU AI/CV modes every 350 ms, two JPEG preview clients, 720p30 RTP and the first minute recorded. All acceptance assertions passed; one encoder generation was preserved throughout.

Observation	Result
Controller run duration	1802.76 seconds
Receiver duration	1803.125 seconds
Decoded frames / rate	54071 / 30.014 FPS
Capture minimum, including startup	29.6 FPS
Capture minimum after first 10 seconds	29.9 FPS
Receiver RTP packets	803104
Receiver bytes	927445056
Lost / late / duplicate RTP packets	0 / 0 / 0
Decoder errors / warnings	0 / 0
Maximum decoded-frame gap	78.651 ms
Maximum sampled Pi temperature	74.9 C
Maximum sampled internal frame age	44 ms
Current firmware thermal flags observed	0 throughout
Local sender drops / malformed frames	0 / 0

Observation	Result
Recorded MP4	1802 decoded frames; complete EOS, zero decoder errors/warnings

The revised receiver socket used an effective 425984-byte receive buffer on this Ubuntu host, with zero socket-buffer drops in sampled `ss` observations. No global kernel buffer tuning was applied. This is an Ethernet acceptance result, not a guarantee for RF or every host workload.

The long run's raw timestamp estimates are **not usable as latency**: clock offset changed by 62.982 ms, exceeding the combined 7.897 ms calibration uncertainty; the raw median was negative. This run validates decode continuity, packet delivery and Pi workload stability. A separate short, clock-qualified measurement is reported below. The public aggregate plot excludes timestamp latency.



Completed 30-minute Pi 5 aggregate measurements. Capture rate and internal frame age do not measure optical latency.

Clock validity and candidate profile

Immediately after reboot, the 18.275-second Pi3 candidate-profile check on Pi5 decoded **235 frames at 15.017 FPS**, 640x360, with zero reported packet loss/errors and an 88.368 ms maximum gap. This validates profile behavior on Pi5 only. Its clocks changed offset by **227.056 ms** during the run, so its negative software latency estimates are invalid and must not be used.

The monitor now rejects latency summaries when before/after clock offset changes exceed combined calibration uncertainty, or when the median estimate is negative. It reports null latency quantiles, a reason and explicitly raw timestamp estimates. Two unit tests cover valid clocks and rejection of drift/negative/missing measurements. Decoder counts/gaps/loss remain independent of clock validity.

Software latency method

The RTP sender stamps each encoded access unit after encoding using an eight-byte Unix timestamp in a private RTP extension. The host maps that timestamp through its jitter/depay/decode pipeline to the decoded RGB frame. Eight HTTP clock samples before/after the run select the smallest-roundtrip estimate and report half-roundtrip uncertainty and observed offset drift.

The coexistence run measured p50 **13.892 ms**, p95 **22.587 ms** from encoder output to host decoded frame. Initial clock uncertainty was several milliseconds (approximately 2.9-5.2 ms in the short first run). These numbers exclude exposure, sensor readout, ISP, H.264 encoding, display queues and monitor refresh. They are **not optical end-to-end latency** and should not be compared directly with glass-to-glass measurements.

`/latency-target` supplies a changing browser timer for a later physical test. It neither measures latency automatically nor overcomes refresh/scheduling limitations. No optical result is claimed.

Additional relay regression: retained failures

The first final relay check decoded 241 frames and observed a 5032.706 ms maximum gap; its receiver reported 1052 lost packets while the relay intentionally dropped 915. It failed acceptance. The relay still used a default receive buffer, so the relay was updated to request 212992 bytes (425984 effective on this host) and report Linux `SO_RXQ_OVFL` counters.

The next run recovered with **607 frames**, a **2318.179 ms** maximum gap and no decoder errors/warnings. Relay receive/send drops were zero. However, the strict loss-accounting assertion still failed: the receiver reported **933 lost packets** against **932 intentional relay drops**. That one additional loss was not attributed; the run is retained as a failed strict accounting check, even though picture recovery succeeded. It does not invalidate the separate direct Ethernet soak's zero-loss result.

Final interruption fix and accounting

A confirmation with an independent RTP input sequence counter reproduced the one extra gap (938 input gaps against 937 intentional drops, no reordering/duplicates). This ruled out a discrepancy confined to the jitterbuffer statistic. Removing the ingress queue's **time** limit, while retaining its packet and byte limits, eliminated that extra gap in the targeted check. The initial queue's elapsed-time accounting across a blackout was therefore implicated; the test's exact loss-accounting assertion was retained.

The final controlled-blackout test passed: **637 decoded frames**, maximum gap **2309.399 ms**, **951 intentional relay drops = 951 observed input sequence gaps = 951 jitterbuffer lost packets**, zero relay receive/send drops and zero decoder errors/warnings. The decoder recovered without restart. The queue remains leaky and bounded at 256 packets / 307200 bytes; `max-size-time=0` prevents the idle interval itself from acting as queue occupancy. Jitter waiting remains bounded at 30 ms.

The earlier 30-minute direct soak used the initial 100 ms queue-time limit. The final queue change targets interruption handling; a separate direct minute-long regression covers the final configuration. Capture, Pi encoding and the successful direct-run results are preserved. A physical RF interruption is still untested.

The sequence diagnostic records forward gaps separately from reorders and duplicates. Later reordered arrivals do not subtract earlier forward gaps; interpret them as missing packets only in an ordered test such as this one. GStreamer remains responsible for runtime RTP reordering.

Final direct regression after the queue fix

A final **62.80-second** controller run exercised the final queue configuration, all four AI modes, direct RTP and ten seconds of concurrent recording. It passed all assertions: **1851 decoded frames / 29.578 FPS**, capture minimum **29.9 FPS**, zero input sequence gaps/reordering/duplicates, zero jitterbuffer lost/late packets and zero decoder errors/warnings. Maximum temperature was **72.7 C**, internal frame age **27 ms**, one encoder generation throughout.

The maximum decoded-frame gap was **969.520 ms**, below the test's 1500 ms acceptance limit, but substantially worse than the longer run's 78.651 ms. This test included recording start/stop; the exact cause of the short pause was not profiled. Do not infer uninterrupted low-latency delivery from the median alone. A strict real-time application still needs profiling of capture/record-finalization stalls and its own tighter acceptance budget.

Clock-qualified software latency was **p50 11.125 ms / p95 22.404 ms**, with **0.128 ms** observed offset drift and before/after uncertainties **3.266 / 3.793 ms**. This excludes the same exposure/ISP/encoding/display stages as all other software timestamp measurements.

Final browser and clock-qualified check

The host browser's **Start RTP** and **Stop RTP** controls were exercised against the real board. Start locked incompatible controls and began sending; Stop released them. The receiver decoded **393 frames at 30.103 FPS**, zero loss/errors/warnings, with a 73.430 ms maximum gap. Its 25.083-second process duration includes waiting for the manual browser start; the 11.950-second first-frame time is not a camera startup benchmark.

This short check passed clock qualification: offset changed by **0.495 ms**, with before/after uncertainties **2.689 / 3.114 ms**. Encoder-output-to-decoded-frame latency was **p50 11.042 ms, p95 23.654 ms**. All four AI modes remained enabled. These are software transport/decode estimates with millisecond clock uncertainty; exposure, ISP, encoding and display remain excluded.

Reproduce

Run host unit tests without the board:

```
python3 -m unittest discover -s tests -p 'test_*.py' -v
```

The suite has 19 tests, including wire-level FU-A reconstruction, sequence wrap/marker/timestamp validation, address boundaries, and the custom sender decoded by standard GStreamer. Synthetic test media are generated locally.

With the real Pi deliberately selected and no recording/RTP session active:

```
/usr/bin/python3 tests/transport_soak.py --seconds 1800 --record-seconds 60
python3 tests/hardware_check.py
/usr/bin/python3 tests/transport_recovery.py
```

The aggregate plot can be regenerated with distribution Matplotlib (optional documentation dependency):

```
python3 tools/plot_transport_report.py artifacts/YOUR_PREFIX-results.json \
  --output artifacts/transport-metrics.png
```

The plot includes only capture rate, temperature, internal frame age and cumulative RTP byte rate. It contains no camera imagery, addresses or runtime secrets. Capture/frame-age samples are polled approximately every five seconds; a brief event between polls can be missed. Receiver gap/loss measurements are tracked separately at every decoded frame/packet.

These hardware scripts change settings and create private media. The transport soak enables four CPU AI/CV modes every 350 ms, previews JPEG, sends 720p30 H.264 over RTP and records the first minute. It verifies one encoder generation, steady capture/decoded rates above 24 FPS, zero reported packet loss/local drops, and a maximum decode gap below 1500 ms. It restores previous settings/analysis afterward. Receiver measurements and errors are retained even when assertions fail.

Remaining physical acceptance

Actual radio adapters, driver/kernel matching, RF capacity/FEC, range, interference, optical latency, cold boot after power removal, Pi 3 A+ memory/hardware encoding, power quality and long-duration 1080p remain unverified. The approximate budget tool is not a PHY-rate or range predictor. Clean shutdown is not protection against abrupt power loss.

Installation bundle and boot strategy

What is packaged

`tools/build_release.py` builds a deterministic gzip/tar **software archive** from the same allowlist as the public release. It includes the application, model, tests, tools, Markdown, PDF and SHA-256 source manifest. Tokens, SSH files, recordings, imported inputs, runtime state and temporary outputs are excluded.

```
python3 tools/build_release.py --output releases/CameraDrone-software.tar.gz
(cd releases && sha256sum -c CameraDrone-software.tar.gz.sha256)
```

The subshell verifies the checksum from the archive's directory. Shell helpers receive canonical executable permissions (0755); other files use 0644. Given unchanged input files, two builds produce identical archive bytes. The PDF itself records its generation date; rebuild the archive after deliberately rebuilding the PDF.

This archive is **not a bootable Raspberry Pi OS image**. It contains no partition table, kernel/root filesystem, EEPROM update, account password or SSH host keys. A cloned working SD card would also clone private credentials and recordings; it is intentionally not the public installation artifact.

Verified archive smoke test

A clean archive was extracted into a temporary directory on Ubuntu and started on a separate loopback port. Simulator status, JPEG output, the latency-target route, newly generated private runtime permissions (0600), executable shell modes (0755) and clean SIGTERM shutdown were verified. This did not contact the Pi or alter the active host service. It validates the software archive route, not a fresh Raspberry Pi OS installation or a bootable image.

Recommended existing-board route

Keep the validated bootable microSD and use the Ubuntu host deployment:

```
export PI_TARGET=youruser@192.168.1.20
bash deploy-studio.sh
```

Dependencies must already exist on the Pi. The helper verifies the model, pairs the private token, copies `camera_app/` to the Pi user's home and installs its generated user service. Stop recording and RTP before deploying. The service uses orderly SIGTERM shutdown and a mixed process-group stop policy for encoder finalization.

Fresh Pi route

1. Write a supported Raspberry Pi OS Lite image using Raspberry Pi Imager. Set your own account, SSH and network configuration. This step needs a boot medium; Ethernet alone is not an installed OS.

2. Extract the software archive under the normal Pi user's home. In the extracted `CameraDrone` directory, run the check and explicitly install dependencies if needed:

```
/usr/bin/python3 tools/setup_pi.py --check
/usr/bin/python3 tools/setup_pi.py --install-packages
```

3. Start the Ubuntu host once to generate its private `camera_app/runtime.json`. Copy that file over verified SSH to the extracted Pi `CameraDrone/camera_app/` directory. Do not put it in a public archive or repository.

4. As the normal Pi user, install the service and boot startup:

```
/usr/bin/python3 tools/setup_pi.py --install-service
sudo loginctl enable-linger "$USER"
```

```
systemctl --user status camera-backend.service
```

The installer generates paths for the extracted checkout and its system interpreter. It refuses to provision a non-Pi or unsupported OS and refuses to install the camera user service as root. `--check` only reads board/OS/dependency information. Package installation uses OS repositories, not a downloaded radio-driver installer.

The archive route installs from `~/CameraDrone/camera_app`; the host deployment route uses `~/camera_app`. Choose one managed installation at a time so only one backend owns the CSI camera.

microSD, USB and network boot

The validated deployment boots from microSD. The same software can be deployed onto an OS booted from USB, but USB boot order, storage, power and a physical cold boot must be verified on the actual board. No attached USB device is formatted by these tools. Do not overwrite a flash drive containing another project's image.

A board without microSD/USB requires correctly configured network boot plus DHCP/TFTP/root filesystem services. This release does not provision that boot infrastructure. The historical passive network diagnostic does not make a complete netboot server.

Reproducibility metadata

`tools/environment.py` exports OS, kernel, architecture, selected package versions and source/model hashes. It intentionally omits usernames, hostnames, network addresses, board serials, camera pixels and runtime credentials.

```
python3 tools/environment.py --output artifacts/environment.json
```

The verified Pi 5 used Debian 13.7, kernel `6.18.50+rpt-rpi-2712`, Python 3.13.5, Picamera2 0.3.37 and OpenCV 4.10.0. This records the tested platform; it is not a promise that arbitrary future OS/kernel versions behave identically.

Shutdown

Stop recording and RTP first, then use `sudo systemctl poweroff` on the Pi. A clean software halt is different from physically removing supply power. Never remove a mounted boot medium from a running system. The host simulator/replay can continue while the board is off.

Hardware return and Pi 3 A+ migration

Regression after future Pi 5 changes

The current version has completed functional, ten-minute preview and corrected 30-minute Ethernet RTP regression; see the measured report. Real-camera RTP is integrated; see the transport report. Physical cold boot and radio trials remain separate milestones. Repeat these checks after future changes.

1. Preserve its existing bootable card and deployed baseline. Back up private settings/media separately; they are not in the public repository.

2. Verify the detected sensor (`rpicam-hello --list-cameras`) and confirm no second capture process.

3. Deploy the shared API refactor, select the explicit Pi endpoint and repeat `tests/hardware_check.py`.

4. Verify resolution, exposure/gain/AWB, AF states, zoom, Pi5 processing HDR, full photo, recording and overlays. Compare MP4 decode counts and sidecar data.

5. Repeat the ten-minute soak, watchdog/process recovery and source reconnect tests. Check the new host presets and switching back from Simulator to hardware.

6. Test a clean shutdown and physical cold boot. Previous software reboot success does not cover power removal.

7. Repeat the single-encoder RTP/recording tests on Ethernet before any RF deployment.

Pi 3 Model A+ constraints

The Pi 3 A+ has **512 MB RAM**, one USB 2.0 port and no onboard Ethernet connector. A USB radio, optional Ethernet adapter and power must fit that topology; a powered hub may be needed for development. The CSI ribbon/connector differs from Pi 5's connector, so select the correct cable. Hardware details: official Pi 3 A+ product page and product brief.

The supplied `pi3a-light` profile requests 640x360 at 15 FPS, 5 FPS JPEG preview, 1.5 Mbit/s recording, HDR off and motion-only analysis every 1200 ms. These are candidate settings, **not a validated port**. Background motion itself is unsuitable for a moving drone until camera-motion compensation is designed.

Use Raspberry Pi OS Lite and verify camera support on that OS. Keep the host UI and batch analysis on Ubuntu. Avoid loading the neural network on the Pi if the selected profile does not need it; the current Vision constructor still loads it eagerly, so profile reduction alone does not fully reduce memory. A lazy model loader is a later board-measured optimization. Set explicit memory budgets before enabling object detection.

Pi 5 H.264 encoding is software-based; Pi 3 has a different hardware encoder path. The present engine needs platform-specific encoder selection and testing on Pi 3, not just profile changes. The official camera software guide describes platform-dependent encoding, and Pi 5 encoding performance explains its software encoder.

Definition of a validated port

• Sensor detected and correct controls/capabilities shown; Pi5-only HDR hidden/rejected as appropriate.

• Sustained capture/encoding at the candidate resolution without OOM or excessive swap.

• Actual memory, CPU, temperature, dropped frames and bitrate reported for capture + radio + chosen AI.

• Correct hardware encoder selected; one camera owner; bounded stream/record queues.

- At least 30-minute soak, interruption/recovery, clean shutdown and cold boot.
- Exact OS/kernel/driver/cable/power/adapters documented with measured results.

Later product work

A repeatable boot image, network boot packaging, labeled field datasets, camera calibration, exposure/AF/HDR optical comparisons, adaptive stream settings and flight integration remain separate milestones. A USB boot image should be created from a validated system; this offline work does not claim an installable Pi OS image.

Third-party components

CameraDrone's own source and documentation are MIT licensed. Third-party software and model assets retain their own licenses.

Component	Role	Upstream / license
MobileNet-SSD model and Caffe graph	CPU object detection	chuanqi305/MobileNet-SSD, MIT; full notice in `camera_app/models/LICENSE.MobileNet-SSD`
OpenCV	DNN, Haar faces, QR, motion, tracking, file decoding	OpenCV; installed separately, version dependent
Haar face cascade	Face locations without identification	Distributed with installed OpenCV/opencv-data; retain upstream cascade notice
Picamera2 / libcamera	Pi camera and ISP	Picamera2, BSD-2-Clause; libcamera, component-specific licenses
Flask / Werkzeug	Local HTTP services	Pallets, BSD-3-Clause
Requests	Host-to-camera HTTP client	Requests, Apache-2.0
NumPy	Image arrays and numeric summaries	NumPy, BSD-3-Clause
GStreamer / x264 / FFmpeg	Video encoding, transport, decoding	GStreamer licensing; plugins/codecs have separate licenses, including GPL components
WFB-ng / OpenHD	Future optional radio transport	External projects, not bundled or installed by CameraDrone; consult their licenses
ReportLab / DejaVu fonts	PDF build tools	Installed separately; PDF build script uses system DejaVu fonts

Model inputs and SHA-256 checksums are recorded in `camera_app/models/README.md` and `camera_app/fetch_models.py`. The included weights are about 23 MB. No cloud model or API key is needed.

`tools/fetch_test_fixtures.py` optionally fetches upstream sample images for the historical known-image tests. CameraDrone does not redistribute these photographs or claim their rights. Generated unit-test images and the documentation screenshot contain no real camera imagery.

Public repository publication

The public repository is MaksimNuzhdin/CameraDrone. Publish updates to its existing history. No runtime credentials, recordings or workstation-specific state should be published.

`tools/prepare_public_release.py` creates an allowlisted checkout outside the active workspace, checks it for the actual private runtime token and common GitHub/private-key patterns, and writes a SHA-256 manifest. It includes application source, pinned model weights and license, generated documentation screenshot, tests, CI and the final PDF. It excludes runtime state, inputs, real imagery, upstream sample photographs, logs, caches and build intermediates. Review the resulting manifest before publication.

```
python3 tools/prepare_public_release.py /tmp/cameradrone-public
```

Review and copy the prepared files into an authenticated existing checkout, preserving `.git`. Verify `git diff --check`, inspect the diff, commit and push `main`. Do not initialize a second unrelated history for an existing repository.

The final prepared checkout may already have a commit; do not recreate it unnecessarily. A local `releases/CameraDrone-public.bundle` can preserve the complete clean commit history without including private working data:

```
git clone /path/to/CameraDrone-public.bundle CameraDrone-public
```

Authenticate using the owner's normal GitHub browser/SSH/CLI flow. Never commit credentials. With an existing checkout:

```
git diff --check
git add .
git commit -m 'Add validated Pi camera RTP and installation tooling'
git push origin main
git bundle create /path/to/CameraDrone-public.bundle --all
```

Do not force-push over existing content. Check the resulting repository's public visibility, commit, model asset, PDF and Actions result after publication. A local commit or bundle is not proof of a remote push. Build the optional deterministic software archive with `tools/build_release.py` after generating the final PDF; the archive itself is a local release artifact, not committed recursively into its own contents.